

```

1: `timescale 1ns / 1ps
2: //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3: // Company:
4: // Engineer:
5: //
6: // Create Date: 05/17/2026 01:27:55 PM
7: // Design Name:
8: // Module Name: apb_uart
9: // Project Name:
10: // Target Devices:
11: // Tool Versions:
12: // Description:
13: //
14: // Dependencies:
15: //
16: // Revision:
17: // Revision 0.01 - File Created
18: // Additional Comments:
19: //
20: //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21:
22:
23: module apb_uart(
24:     //APB bus interface
25:     input pclk,
26:     input presetn, // APB reset os traditionally active-low
27:     input [31:0] paddr,
28:     input psel, //1 whe the address decoder selects this uart
29:     input penable,
30:     input pwrite,
31:     input [31:0] pwrdata,
32:     output reg [31:0] prdata,
33:     output pready,
34:
35:     //Physical wire
36:     input rx_pin,
37:     output tx_pin
38: );
39:
40: //Internal wire connecting to raw uart
41: wire uart_reset;
42: reg tx_start;
43: reg [7:0] tx_data_in;
44: wire tx_active;
45: wire tx_done;
46:
47: wire [7:0] rx_data_out;
48: wire rx_done;
49:
50: // Convert APB active-low reset to your UART's active-high reset
51: assign uart_reset = ~presetn;
52:
53: // --- 1. Instantiate YOUR Raw UART Core (Formerly top.v) ---
54: uart_core my_raw_uart (
55:     .clk(pclk),
56:     .reset(uart_reset),
57:
58:     // TX Ports
59:     .tx_start(tx_start),
60:     .tx_data(tx_data_in),
61:     .tx_active(tx_active),
62:     .tx_done(tx_done),
63:     .tx_pin(tx_pin),
64:
65:     // RX Ports
66:     .rx_pin(rx_pin),
67:     .rx_data(rx_data_out),
68:     .rx_done(rx_done)
69: );
70:
71: //APB Write logic( CPU -> UART)
72: always @(posedge pclk) begin
73:     if(!presetn) begin
74:         tx_start <= 1'b0;
75:         tx_data_in <= 8'h00;
76:     end
77:     else begin
78:         //Default: do not start a transmission
79:         tx_start <= 1'b0;
80:
81:         //If bus is selected, enabled and wrirint to TX Register(Offset 0x00)
82:         if(psel && penable && pwrite && paddr[7:0] == 8'h00) begin
83:             tx_data_in <= pwrdata[7:0];
84:             tx_start <= 1'b1; // Send a 1-cycle pulse to start transmission
85:         end
86:     end
87: end
88:
89: // 3. Sticky RX Data Ready Flag --
90: // catches the 1-cycle tx_done pulse and holds it until the CPU reads the data
91: reg rx_data_ready;

```

```

92:
93:     always @(posedge pclk) begin
94:         if(!presetn) begin
95:             rx_data_ready <= 1'b0;
96:         end
97:         else if (rx_done) begin
98:             rx_data_ready <= 1'b1; // New data arrived, flag goes high
99:         end
100:        else if (psel && penable && !pwrite && paddr[7:0] == 8'h04) begin
101:            rx_data_ready <= 1'b0; // CPU just read the RX register, clear the flag.
102:        end
103:    end
104:
105:    //4, APU Read Logic (UART -> CPU)
106:    always @(*) begin
107:        prdata = 32'h0; // Default to 0
108:        if(psel && !pwrite) begin
109:            case(paddr[7:0])
110:                8'h04: prdata = {24'h0, rx_data_out}; //Read rx data
111:                8'h08: prdata = {30'h0, rx_data_ready, tx_active}; // read statis
112:                default: prdata = 32'h0;
113:            endcase
114:        end
115:    end
116:
117:    // --- 5. Ready Logic
118:    //Out registers respond instantly in 1 clock cycle, sp we are always ready
119:    assign pready = 1'b1;
120:
121: endmodule
122:

```